МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ №5

з дисципліни **"Функціональне програмування"**

на тему **"ЛІНИВІ ОБЧИСЛЕННЯ: ГЕНЕРАТОРИ ТА ITERTOOLS"**

Варіант 13. Ковзні максимуми

Виконав:
Студент групи ФЕП-23
Маньківський Максим Володимирович

Перевірив:
проф. Свелеба С. А.

Львів-2026

Мета: зрозуміти генераторні вирази, `yield` / `yield from`, ітератори та лінивість обчислень; навчитися працювати з нескінченними послідовностями й будівельними блоками `itertools`; реалізувати нескінченний потік спостережень, ковзні вікна та ліниве обчислення максимуму, мінімуму й розмаху по вікнах.

Теоретичні відомості:

Ітерований об'єкт — це те, по чому можна пройтися циклом `for`; ітератор — об'єкт зі станом, який на кожному кроці віддає наступне значення і зупиняється через `StopIteration`. Генератори (функції з `yield` та генераторні вирази) створюють ітератори ліниво: кожне значення обчислюється лише тоді, коли його запитує споживач. Це економить пам'ять і дозволяє описувати потенційно нескінченні джерела — наприклад, потік чисел Фібоначчі або потік показів датчика — за умови, що конвеєр завершується обмежувачем `islice`, `takewhile` чи власним `take`.

Модуль `itertools` дає ефективні «цеглини» для лінивих конвеєрів: `islice` для підрізання, `takewhile` / `dropwhile` для умовного відбору, `accumulate` для префіксних згорток, `count` / `cycle` / `repeat` як нескінченні джерела, `starmap` для застосування функції до кортежів. Ковзне вікно над потоком реалізується через `deque(maxlen=k)`: черга зберігає лише `k` останніх елементів і сама виштовхує найстаріший, тому вікна формуються без матеріалізації всього потоку. Генератор вичерпується після одного проходу, тому для повторної ітерації його створюють заново.

Хід роботи:

1. У `streams.py` реалізував базові ліниві потоки з методички: нескінченний `fib_stream`, `naturals` через `yield from count`, утиліти `take` (єдина матеріалізація у список) і `drop` (ліниве «промотування» через `islice`), а також

`sliding_window` на `deque(maxlen=k)`, що не віддає жодного вікна, якщо елементів менше `k`, і піднімає `ValueError` для `k <= 0`:

```python
def fib_stream() -> Iterator[int]:
    """Нескінченний потік чисел Фібоначчі: 0, 1, 1, 2, 3, ..."""
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b


def naturals(start: int = 1) -> Iterator[int]:
    """Нескінченний лічильник натуральних чисел від start (делегує itertools.count)."""
    yield from count(start)


def take(n: int, it: Iterable[T]) -> list[T]:
    """Перші n елементів як список; n < 0 — ValueError, n == 0 — порожній список."""
    if n < 0:
        raise ValueError("n must be >= 0")
    return list(islice(it, n))

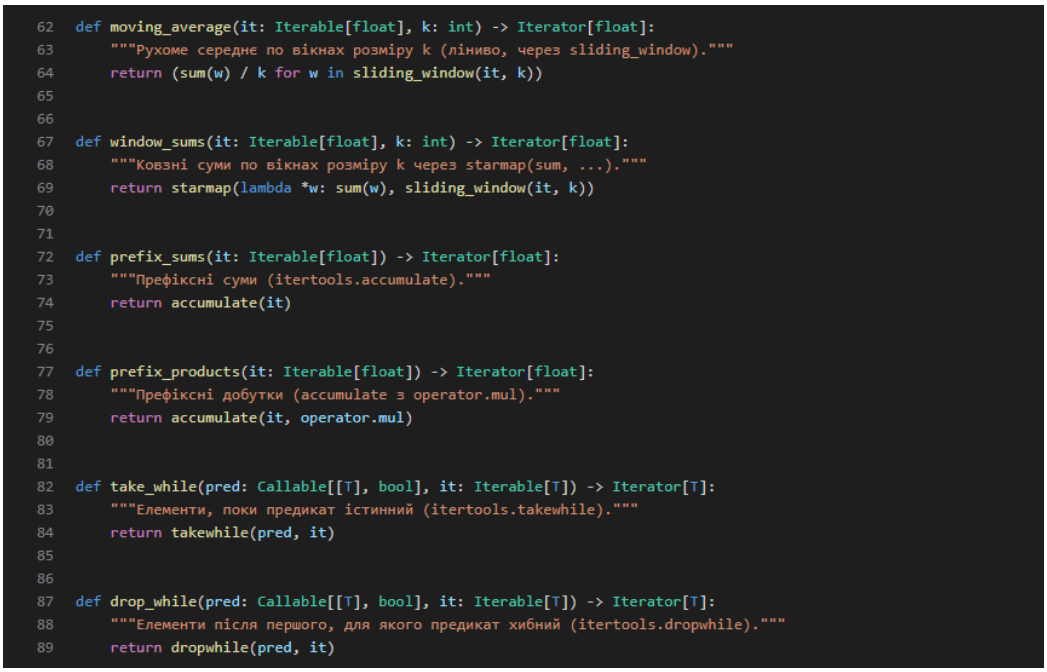

def drop(n: int, it: Iterable[T]) -> Iterator[T]:
    """Ітератор без перших n елементів (ліниво «промотує» n значень)."""
    if n < 0:
        raise ValueError("n must be >= 0")
    iterator = iter(it)
    next(islice(iterator, n, n), None)
    return iterator


def sliding_window(it: Iterable[T], k: int) -> Iterator[tuple[T, ...]]:
    """Ковзні вікна розміру k як кортежі; якщо елементів менше k — жодного вікна."""
    if k <= 0:
        raise ValueError("window size must be > 0")
    iterator = iter(it)
    window: deque[T] = deque(islice(iterator, k), maxlen=k)
    if len(window) < k:
        return
    yield tuple(window)
    for x in iterator:
        window.append(x)
        yield tuple(window)

```

2. Додав похідні ліниві операції над вікнами та обгортки `itertools`: `moving_average` як генераторний вираз над вікнами, `window_sums` через `starmap`,

`prefix_sums` / `prefix_products` через `accumulate` і `take_while` / `drop_while`:



```python
def moving_average(it: Iterable[float], k: int) -> Iterator[float]:
    """Рухоме середнє по вікнах розміру k (ліниво, через sliding_window)."""
    return (sum(w) / k for w in sliding_window(it, k))


def window_sums(it: Iterable[float], k: int) -> Iterator[float]:
    """Ковзні суми по вікнах розміру k через starmap(sum, ...)."""
    return starmap(lambda *w: sum(w), sliding_window(it, k))

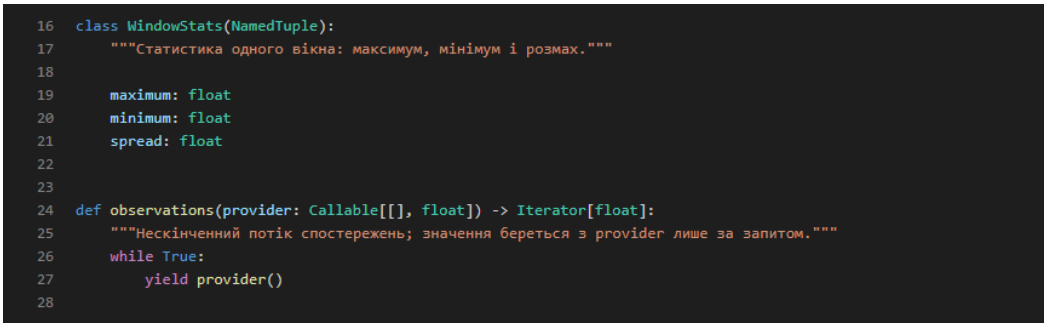

def prefix_sums(it: Iterable[float]) -> Iterator[float]:
    """Префіксні суми (itertools.accumulate)."""
    return accumulate(it)


def prefix_products(it: Iterable[float]) -> Iterator[float]:
    """Префіксні добутки (accumulate з operator.mul)."""
    return accumulate(it, operator.mul)


def take_while(pred: Callable[[T], bool], it: Iterable[T]) -> Iterator[T]:
    """Елементи, поки предикат істинний (itertools.takewhile)."""
    return takewhile(pred, it)


def drop_while(pred: Callable[[T], bool], it: Iterable[T]) -> Iterator[T]:
    """Елементи після першого, для якого предикат хибний (itertools.dropwhile)."""
    return dropwhile(pred, it)
```

3. За варіантом 13 у `observations.py` описав результат одного вікна `WindowStats` (`NamedTuple` з максимумом, мінімумом і розмахом) і нескінченний генератор спостережень `observations(provider)`, який викликає інжектованого постачальника значень лише за запитом споживача:



```python
class WindowStats(NamedTuple):
    """Статистика одного вікна: максимум, мінімум і розмах."""

    maximum: float
    minimum: float
    spread: float


def observations(provider: Callable[[], float]) -> Iterator[float]:
    """Нескінченний потік спостережень; значення береться з provider лише за запитом."""
    while True:
        yield provider()

```

4. Реалізував ліниву ковзну статистику: `window_max` і `window_min` як `map(max, …)` / `map(min, …)` над `sliding_window`, `window_range` та `window_stats` як генераторні

вирази, а `first_n_stats` бере перші `n` результатів через `islice` — це єдине місце, де потік матеріалізується:

```python
def window_max(it: Iterable[float], k: int) -> Iterator[float]:
    """Ковзний максимум по вікнах розміру k."""
    return map(max, sliding_window(it, k))


def window_min(it: Iterable[float], k: int) -> Iterator[float]:
    """Ковзний мінімум по вікнах розміру k."""
    return map(min, sliding_window(it, k))


def window_range(it: Iterable[float], k: int) -> Iterator[float]:
    """Ковзний розмах (max - min) по вікнах розміру k."""
    return (max(w) - min(w) for w in sliding_window(it, k))


def window_stats(it: Iterable[float], k: int) -> Iterator[WindowStats]:
    """Ковзна статистика (max, min, range) по вікнах розміру k одним проходом."""
    return (WindowStats(max(w), min(w), max(w) - min(w)) for w in sliding_window(it, k))


def first_n_stats(it: Iterable[float], k: int, n: int) -> list[WindowStats]:
    """Перші n результатів ковзної статистики (єдина матеріалізація — на виході)."""
    if n < 0:
        raise ValueError("n must be >= 0")
    return list(islice(window_stats(it, k), n))
```

5. У `main.py` зробив демонстраційний запуск: детермінований постачальник на `cycle`, перші значення Фібоначчі, `drop` + `take`, віконні суми натуральних чисел, рухоме середнє, префіксні суми та ковзна статистика по нескінченному потоку спостережень:

```python
SAMPLES = [3.0, 1.0, 4.0, 1.0, 5.0, 9.0, 2.0, 6.0]


def make_provider(values: list[float]) -> Callable[[], float]:
    """Детермінований постачальник: циклічно віддає значення зі списку."""
    source = cycle(values)
    return lambda: next(source)


def main() -> None:
    """Друкує приклади роботи потоків (єдиний I/O у лабораторній)."""
    print("fib first 10:", take(10, fib_stream()))
    print("fib drop 5 take 8:", take(8, drop(5, fib_stream())))
    print(
        "window sums of naturals (k=3, first 10):", take(10, window_sums(naturals(), 3))
    )
    print("moving average [1..6], k=2:", list(moving_average([1, 2, 3, 4, 5, 6], 2)))
    print("prefix sums [1,2,3,4]:", list(prefix_sums([1, 2, 3, 4])))

    stream = observations(make_provider(SAMPLES))
    print("window max (k=3, first 6):", list(islice(window_max(stream, 3), 6)))
    print("window range (k=3, first 6):", list(islice(window_range(SAMPLES, 3), 6)))
    for stats in first_n_stats(observations(make_provider(SAMPLES)), 3, 5):
        print(
            f"max={stats.maximum:.1f} min={stats.minimum:.1f} range={stats.spread:.1f}"
        )


if __name__ == "__main__":
    main()
```

6. Написав тести для вікон у `tests/test_sliding_window.py`: базові вікна, суми через `starmap`, рухоме середнє та граничні випадки — порожнє джерело, `k`

більше за довжину, вікно з одного елемента, некоректний розмір вікна:

```python
def test_windows_basic() -> None:
    assert list(sliding_window([1, 2, 3, 4], 3)) == [(1, 2, 3), (2, 3, 4)]


def test_window_sums() -> None:
    assert list(starmap(lambda *w: sum(w), sliding_window(range(5), 2))) == [1, 3, 5, 7]
    assert take(4, window_sums(naturals(), 3)) == [6, 9, 12, 15]


def test_moving_average() -> None:
    assert list(moving_average([1, 2, 3, 4], 2)) == [1.5, 2.5, 3.5]


def test_edge_cases() -> None:
    assert list(sliding_window([], 2)) == []
    assert list(sliding_window([1, 2], 3)) == []
    assert list(sliding_window([1, 2, 3], 3)) == [(1, 2, 3)]
    assert list(sliding_window([7], 1)) == [(7,)]


def test_invalid_window_size() -> None:
    with pytest.raises(ValueError):
        list(sliding_window([1, 2, 3], 0))
    with pytest.raises(ValueError):
        list(sliding_window([1, 2, 3], -2))
```

7. У `tests/test_window_stats.py` перевірив max / min / range на фіксованому наборі, узгодженість `window_stats` з окремими функціями, роботу над нескінченним потоком через `islice` та крайні випадки (`n = 0`, `k = 0`, `n < 0`):

```python
def test_window_max_min_range() -> None:
    assert list(window_max(SAMPLES, 3)) == [4.0, 4.0, 5.0, 9.0, 9.0, 9.0]
    assert list(window_min(SAMPLES, 3)) == [1.0, 1.0, 1.0, 1.0, 2.0, 2.0]
    assert list(window_range(SAMPLES, 3)) == [3.0, 3.0, 4.0, 8.0, 7.0, 7.0]


def test_window_stats_matches_parts() -> None:
    stats = list(window_stats(SAMPLES, 4))
    assert stats[0] == WindowStats(4.0, 1.0, 3.0)
    assert [s.maximum for s in stats] == list(window_max(SAMPLES, 4))
    assert [s.minimum for s in stats] == list(window_min(SAMPLES, 4))


def test_first_n_from_infinite_stream() -> None:
    source = cycle(SAMPLES)
    stream = observations(lambda: next(source))
    assert first_n_stats(stream, 3, 2) == [
        WindowStats(4.0, 1.0, 3.0),
        WindowStats(4.0, 1.0, 3.0),
    ]
    assert list(islice(window_max(observations(lambda: 7.0), 2), 3)) == [7.0, 7.0, 7.0]


def test_edge_cases() -> None:
    assert list(window_max([], 2)) == []
    assert list(window_range([1.0, 2.0], 5)) == []
    assert first_n_stats(SAMPLES, 3, 0) == []
    with pytest.raises(ValueError):
        list(window_min(SAMPLES, 0))
    with pytest.raises(ValueError):
        first_n_stats(SAMPLES, 3, -1)

```

8. Додав три property-тести на Hypothesis: для випадкових списків і розмірів вікна розмах невід’ємний і дорівнює різниці максимуму та мінімуму, кількість

вікон дорівнює `max(0, len - k + 1)`, а ковзний максимум збігається з наївним обчисленням через зрізи:

```python
floats = st.lists(
    st.floats(allow_nan=False, allow_infinity=False, width=32), max_size=40
)


@given(floats, st.integers(min_value=1, max_value=6))
def test_property_range_is_non_negative_and_bounded(xs: list[float], k: int) -> None:
    for s in window_stats(xs, k):
        assert s.spread >= 0
        assert s.minimum <= s.maximum
        assert s.spread == s.maximum - s.minimum


@given(floats, st.integers(min_value=1, max_value=6))
def test_property_window_count(xs: list[float], k: int) -> None:
    expected = max(0, len(xs) - k + 1)
    assert len(list(window_max(xs, k))) == expected
    assert len(list(window_min(xs, k))) == expected


@given(floats, st.integers(min_value=1, max_value=6))
def test_property_matches_naive_slices(xs: list[float], k: int) -> None:
    naive = [max(xs[i : i + k]) for i in range(len(xs) - k + 1)]
    assert list(window_max(xs, k)) == naive
```

9. У `tests/test_laziness.py` підтвердив лінивість: ітератор-лічильник `CountingSource` фіксує, скільки елементів у нього реально запитали, і тести

показують, що `take`, `sliding_window`, конвеєр `moving_average(drop(…))` та генератор `observations` обчислюють рівно стільки значень, скільки спожито:

```python
class CountingSource:
    """Ітератор, що рахує, скільки елементів у нього реально запитали."""

    def __init__(self) -> None:
        self.pulled = 0

    def __iter__(self) -> Iterator[int]:
        return self

    def __next__(self) -> int:
        self.pulled += 1
        return self.pulled


def test_take_pulls_exactly_n() -> None:
    src = CountingSource()
    assert take(4, src) == [1, 2, 3, 4]
    assert src.pulled == 4


def test_sliding_window_pulls_only_what_is_consumed() -> None:
    src = CountingSource()
    windows = sliding_window(src, 3)
    assert src.pulled == 0  # генератор ще не стартував
    assert next(windows) == (1, 2, 3)
    assert src.pulled == 3
    assert next(windows) == (2, 3, 4)
    assert src.pulled == 4


def test_pipeline_is_lazy_end_to_end() -> None:
    src = CountingSource()
    pipeline = moving_average(drop(2, src), 2)
    assert src.pulled == 2  # drop промотує n елементів одразу, вікна ще не стартували
    assert list(islice(pipeline, 2)) == [3.5, 4.5]
    assert src.pulled == 5  # 2 пропущено + 3 для двох вікон по 2


def test_provider_called_on_demand_only() -> None:
    calls = {"n": 0}

    def provider() -> float:
        calls["n"] += 1
        return float(calls["n"])

    stream = window_max(observations(provider), 2)
    assert calls["n"] == 0
    assert list(islice(stream, 3)) == [2.0, 3.0, 4.0]
    assert calls["n"] == 4
```

10. Запустив демонстрацію: усі потоки обмежуються `take` / `islice`, тому нескінченні джерела (Фібоначчі, натуральні числа, циклічний постачальник) обробляються без зависання і без зайвої пам’яті:

```
C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab05> python main.py
fib first 10: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
fib drop 5 take 8: [5, 8, 13, 21, 34, 55, 89, 144]
window sums of naturals (k=3, first 10): [6, 9, 12, 15, 18, 21, 24, 27, 30, 33]
moving average [1..6], k=2: [1.5, 2.5, 3.5, 4.5, 5.5]
prefix sums [1,2,3,4]: [1, 3, 6, 10]
window max (k=3, first 6): [4.0, 4.0, 5.0, 9.0, 9.0, 9.0]
window range (k=3, first 6): [3.0, 3.0, 4.0, 8.0, 7.0, 7.0]
max=4.0 min=1.0 range=3.0
max=4.0 min=1.0 range=3.0
max=5.0 min=1.0 range=4.0
max=9.0 min=1.0 range=8.0
max=9.0 min=2.0 range=7.0
```

11. Перевірив проєкт автоматичними інструментами: усі 26 тестів pytest проходять, `mypy --strict` не знаходить помилок типів, `ruff` та `black` не мають зауважень до стилю:

```
C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab05> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\maksi\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\maksi\Desktop\labs\FunctionalProgramming
configfile: pyproject.toml
plugins: hypothesis-6.170.0, platformdirs-4.12.3
collecting ... collected 26 items

tests\test_accumulate.py::test_accumulate_sum_prod PASSED                [  3%]
tests\test_accumulate.py::test_accumulate_max_and_empty PASSED           [  7%]
tests\test_accumulate.py::test_takewhile_dropwhile_on_infinite PASSED    [ 11%]
tests\test_fib_stream.py::test_first_10 PASSED                           [ 15%]
tests\test_fib_stream.py::test_drop_then_take PASSED                     [ 19%]
tests\test_fib_stream.py::test_stream_is_fresh_each_call PASSED          [ 23%]
tests\test_laziness.py::test_take_pulls_exactly_n PASSED                 [ 26%]
tests\test_laziness.py::test_sliding_window_pulls_only_what_is_consumed PASSED [ 30%]
tests\test_laziness.py::test_pipeline_is_lazy_end_to_end PASSED          [ 34%]
tests\test_laziness.py::test_provider_called_on_demand_only PASSED       [ 38%]
tests\test_sliding_window.py::test_windows_basic PASSED                  [ 42%]
tests\test_sliding_window.py::test_window_sums PASSED                    [ 46%]
tests\test_sliding_window.py::test_moving_average PASSED                 [ 50%]
tests\test_sliding_window.py::test_edge_cases PASSED                     [ 53%]
tests\test_sliding_window.py::test_invalid_window_size PASSED            [ 57%]
tests\test_take_drop.py::test_take_advances_shared_iterator PASSED       [ 61%]
tests\test_take_drop.py::test_drop_then_take_on_count PASSED             [ 65%]
tests\test_take_drop.py::test_edge_cases PASSED                          [ 69%]
tests\test_take_drop.py::test_negative_n_rejected PASSED                 [ 73%]
tests\test_window_stats.py::test_window_max_min_range PASSED             [ 76%]
tests\test_window_stats.py::test_window_stats_matches_parts PASSED       [ 80%]
tests\test_window_stats.py::test_first_n_from_infinite_stream PASSED     [ 84%]
tests\test_window_stats.py::test_edge_cases PASSED                       [ 88%]
tests\test_window_stats.py::test_property_range_is_non_negative_and_bounded PASSED [ 92%]
tests\test_window_stats.py::test_property_window_count PASSED            [ 96%]
tests\test_window_stats.py::test_property_matches_naive_slices PASSED    [100%]

============================= 26 passed in 0.81s ==============================

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab05> python -m mypy .
Success: no issues found in 10 source files

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab05> python -m ruff check .
All checks passed!

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab05> python -m black --check .
All done! ✨ 🍰 ✨
10 files would be left unchanged.
```

Висновок: Під час виконання лабораторної роботи я реалізував ліниві потоки на генераторах та `itertools`: нескінченні `fib_stream`, `naturals` і `observations` з інжектованим постачальником, універсальні `take` / `drop` / `sliding_window` на `deque`, а за варіантом 13 — ковзні максимум, мінімум і розмах через `map` і генераторні вирази над вікнами з відбором перших `N` результатів через `islice`. Жодна функція ядра не матеріалізує потік, що підтверджено тестом лінивості з лічильником звернень; граничні випадки, property-тести на Hypothesis та перевірки mypy, ruff і black підтвердили коректність реалізації.